

Property-Based Testing (PBT)



Introduction

```
1 def test_square():  
2     assert square(0) == 0  
3     assert square(1) == 1  
4     assert square(4) == 16  
5     assert square(-1) == 1  
6     assert isnan(square(nan))
```

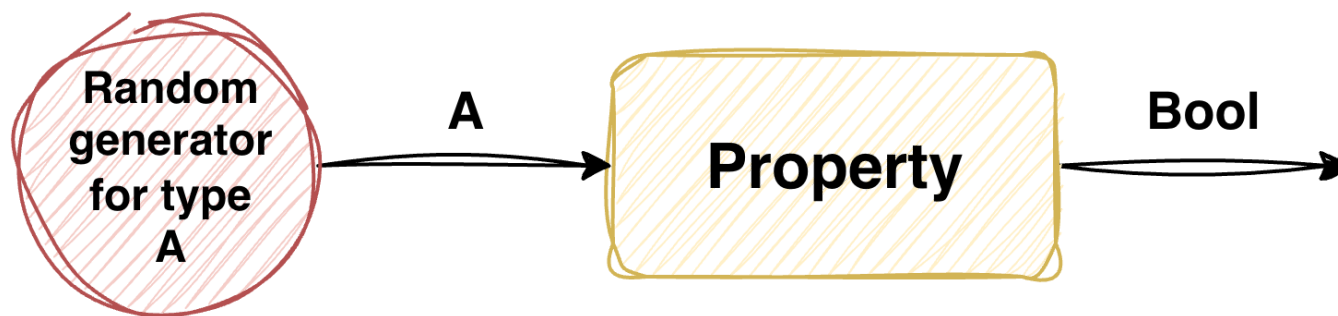
- Does this scale?
- Similar corner cases
 - int: 0, -1, MIN_INT, MAX_INT, ...
 - float: ± 0.0 , $\pm\infty$, NaN, ...
 - lists: [], [1], ...
- Can we make this more systematic / automate it?



Property-based testing basics

- Base tests on properties of our functions:

```
1 def property(l : list[float]) -> bool:  
2   return reverse(reverse(l)) == l
```



- Run arbitrarily large number of tests automatically
- Counter examples $\leftrightarrow$ Shrinking
- Original PBT $\Rightarrow$ QuickCheck (Haskell)

Python **hypothesis** library

- Generators are called **strategies**

```
1 from hypothesis import given
2 from hypothesis.strategies import floats, lists
3
4 @given(lists(floats()))
5 def test_prop(input):
6     assert (reverse(reverse(input)) == input)
```

- Strategies can be adapted:

```
1 @given(floats(allow_infinity = False, allow_nan = False))
2 ...
3
4 @given(integers().filter(lambda n: n != 0))
5 ...
```



Custom strategies

```
1 from hypothesis.strategies import given, integers, composite
2
3 @composite
4 def ordered_pairs(draw):
5     n1 = draw(integers())
6     n2 = draw(integers(min_value=n1))
7     return (n1, n2)
8
9 @given(ordered_pairs())
10 def test_pairs_are_ordered(pair):
11     n1, n2 = pair
12     assert n1 <= n2
```

①

②

③

- ① **draw** allows to *draw* random values from other strategies
- ② **n2** depends on the value of **n1**
- ③ Can use custom strategies like built-in one



What makes a *good* property?

- Avoid replicating implementation

```
1 def max(a : int, b : int) -> int:
2     if a < b:
3         return b
4     else: return a
5
6 @given(integers(), integers())
7 def test_max(a, b):
8     res = max(a, b)
9     assert b if a < b else a
```

```
1 def test_max(a, b):
2     res = max(a, b)
3     assert res >= a and res >= b
```

- ties in with *differential testing*



- Think in terms of **postconditions**
 - Relationship between inputs and outputs
 - What is true after the function returns?

```
1 @given(lists(floats()))
2 def test_prop(input):
3     res = sort(input)
4     assert len(input) == len(res)
```

- Algebraic properties: Involution, Commutativity, Associativity, Unitality?

```
1 @given(integers(), integers())
2 def test_max(a, b):
3     res1 = max(a, b)
4     res2 = max(b, a)
5     assert res1 == res2
```



- Metamorphic properties
 - Often informed by the domain
 - Conservation of mass/momentum/energy, ...
 - Invariances
 - Symmetries



Final thoughts

- PBT is very powerful
 - used in many different domains: telecommunication software, election software, compilers, language runtimes, ...
 - easy to use in CI (regression testing)
- Should be combined with other approaches
- Custom generators + shrinkers (active research)
- Frameworks exists in most languages



Thank you!

Happy testing 😊

